# RISC-V PMP 偵錯

原創 qinyunti 嵌入式 Lee 2023 年 06 月 08 日 11:49 湖南

## PMP 導致異常問題分析

## 現象和確認異常原因

觸發了異常中斷
查看異常原因為 7 store access-fault exception.
並查看異常訪問的地址為 0x28382ad0

```
(gdb) info reg mcause
mcause         0x7      0x7
(gdb) info reg mtval
mtval          0x28382ad0       0x28382ad0
(gdb)
```

gdb 也確認這個地址確實只能讀不能寫

```
(gdb) set *(unsigned int*)0x28382ad0=0xa5a5a5a5
Cannot access memory at address 0x28382ad0
```

```
(gdb) x /1xw 0x28382ad0
0x28382ad0:     0xa5[illegible]
```

## 查看 PMP 暫存器確認根本原因

```
(gdb) info reg pmpcfg0
pmpcfg0        0x0      0x0
(gdb) info reg pmpcfg1
pmpcfg1        0x0      0x0
(gdb) info reg pmpcfg2
pmpcfg2        0x0      0x0
(gdb) info reg pmpcfg3
pmpcfg3        0x5100   0x5100
(gdb)
```

Pmpcfg0 對應地址 pmp0cfg~pmp3cfg
Pmpcfg1 對應地址 pmp4cfg~pmp7cfg
Pmpcfg2 對應地址 pmp8cfg~pmp11cfg
Pmpcfg3 對應地址 pmp12cfg~pmp15cfg
pmpcfg3 為 0x5100 即對應 pmp13cfg 為 0x51
8 位的組態屬性 0x51 如下　唯讀
A=10 即 NA4 表示 4 位元組區域，即 pmpaddrxx 對應的 4 位元組區域

| pmpaddr | pmpcfg.A | Match type and size |
|---|---|---|
| yyyy...yyyy | NA4 | 4-byte NAPOT range |
| yyyy...yyy0 | NAPOT | 8-byte NAPOT range |
| yyyy...yy01 | NAPOT | 16-byte NAPOT range |
| yyyy...y011 | NAPOT | 32-byte NAPOT range |
| ... | ... | ... |
| yy01...1111 | NAPOT | $2^{XLEN}$-byte NAPOT range |
| y011...1111 | NAPOT | $2^{XLEN+1}$-byte NAPOT range |
| 0111...1111 | NAPOT | $2^{XLEN+2}$-byte NAPOT range |
| 1111...1111 | NAPOT | $2^{XLEN+3}$-byte NAPOT range |

L=1
A=10=2 NA4
R=1
X=0
W=0

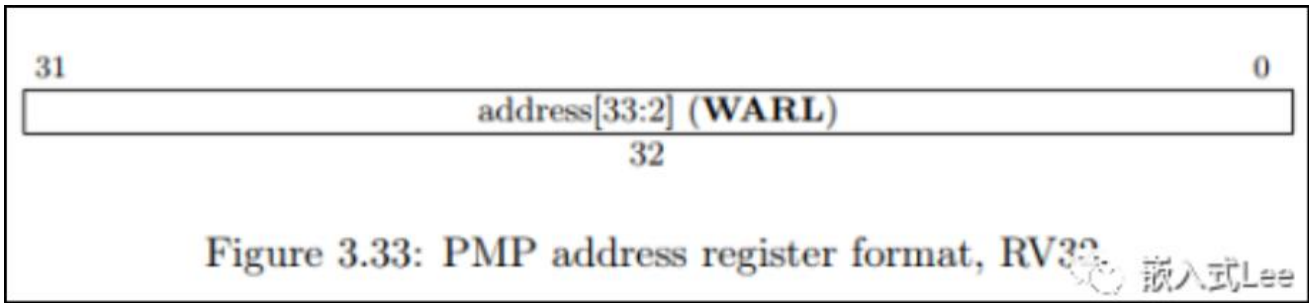
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| L (WARL) | 0 (WARL) | | A (WARL) | | X (WARL) | W (WARL) | R (WARL) |
| 1 | 2 | | 2 | | 1 | 1 | 1 |

Figure 3.35: PMP configuration register format.

| A | Name | Description |
|---|---|---|
| 0 | OFF | Null region (disabled) |
| 1 | TOR | Top of range |
| 2 | NA4 | Naturally aligned four-byte region |
| 3 | NAPOT | Naturally aligned power-of-two region, ≥8 bytes |

Table 3.10: Encoding of A field in PMP configuration register

對應 pmp13cfg 的 pmpaddr13=0xa0e0ab4 是 34 位地址右移 2 位的值，所以左移兩位恢復實際值是 0x2C382AD0




表示的範圍是[0x2C382AD0, 0x2C382AD3] 4 位元組

```
(gdb) info reg pmpaddr0
pmpaddr0       0x0      0x0
(gdb) info reg pmpaddr1
pmpaddr1       0x0      0x0
(gdb) info reg pmpaddr2
pmpaddr2       0x0      0x0
(gdb) info reg pmpaddr3
pmpaddr3       0x0      0x0
(gdb) info reg pmpaddr4
pmpaddr4       0x0      0x0
(gdb) info reg pmpaddr5
pmpaddr5       0x0      0x0
(gdb) info reg pmpaddr6
pmpaddr6       0x0      0x0
(gdb) info reg pmpaddr7
pmpaddr7       0x0      0x0
(gdb) info reg pmpaddr8
pmpaddr8       0x0      0x0
(gdb) info reg pmpaddr9
pmpaddr9       0x0      0x0
(gdb) info reg pmpaddr10
pmpaddr10      0x0      0x0
(gdb) info reg pmpaddr11
pmpaddr11      0x0      0x0
(gdb) info reg pmpaddr12
pmpaddr12      0x0      0x0
(gdb) info reg pmpaddr13
pmpaddr13      0xa0e0ab4        0xa0e0ab4
(gdb) info reg pmpaddr14
pmpaddr14      0x0      0x0
(gdb) info reg pmpaddr15
pmpaddr15      0x0      0x0
(gdb) info reg pmpaddr16
Invalid register `pmpaddr16'
(gdb)
```

我們的地址空間是 0x28000000-0x2bffffff
寫 0x2C38 2AD0 實際對應到了寫 0x2838 2AD0 64M 繞回。
所以寫 0x2838 2AD0 這個不具備寫屬性的地址觸發了 store access-fault exception.異常

所以以上確認了原因是 PMP 設定了該地址不具備寫屬性，而程式碼中去寫所以導致了異常。

# 程式碼

我們的空間最多隻有 128M，最多到 0x2bff_ffff，為什麼寫了個 0x2C38_2AD0 的地址進去。先找到對應的程式碼，由於是用的所以 13 的組態，所以先找這個索引對應

```
#define PMP_FIXED_INDEX_TASK_STACK 13
```

然後再搜尋這個索引
找到如下函數

```
void pmp_task_stack_set(uint32_t addr)
{
    uint32_t address = (addr + 3) >> 2;
    uint32_t flags = PMP_NA4 | PMP_M | PMP_R;

    /* clear */
    write_pmpcfg(PMP_FIXED_INDEX_TASK_STACK, 0);

    /* index 13 */
    PMP_WRITE_CSR(PMP_FIXED_INDEX_TASK_STACK, address);
    write_pmpcfg(PMP_FIXED_INDEX_TASK_STACK, flags);
}
```

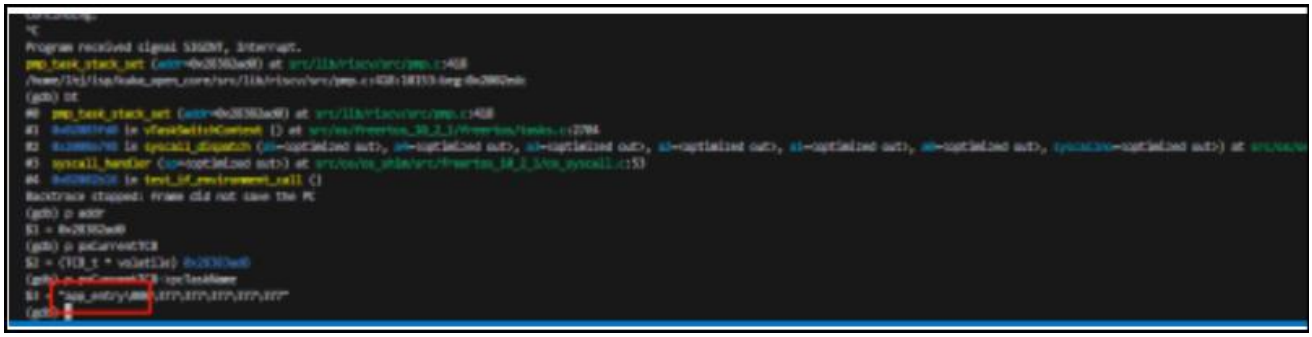
這個函數在 vTaskSwitchContext 呼叫

```
    if (prev != pxCurrentTCB) {
        pmp_task_stack_set((uint32_t)pxCurrentTCB->pxStack);
    }
```

即任務有切換時，設定棧底 4 位元組不可寫，用於棧溢出檢測。加個死循環用於在設定該值時停住

```
void pmp_task_stack_set(uint32_t addr) IRAM_TEXT(pmp_task_stack_set);
void pmp_task_stack_set(uint32_t addr)
{
    uint32_t address = (addr + 3) >> 2;
    uint32_t flags = PMP_NA4 | PMP_M | PMP_R;
    if(0xa0e0ab4 == address)
    {
        while(1){}
    }
    /* clear */
    write_pmpcfg(PMP_FIXED_INDEX_TASK_STACK, 0);

    /* index 13 */
    PMP_WRITE_CSR(PMP_FIXED_INDEX_TASK_STACK, address);
    write_pmpcfg(PMP_FIXED_INDEX_TASK_STACK, flags);
}
```



所以要確認什麼時候寫了 0x2838_2AD0 這個地址，由於使能了 PMP，所以 watchpoint 抓不到，先觸發了 pmp 的異常。所以先關閉 pmp 設定

```
        if (prev != pxCurrentTCB) {
            ///pmp_task_stack_set((uint32_t)pxCurrentTCB->pxStack);
        }
```

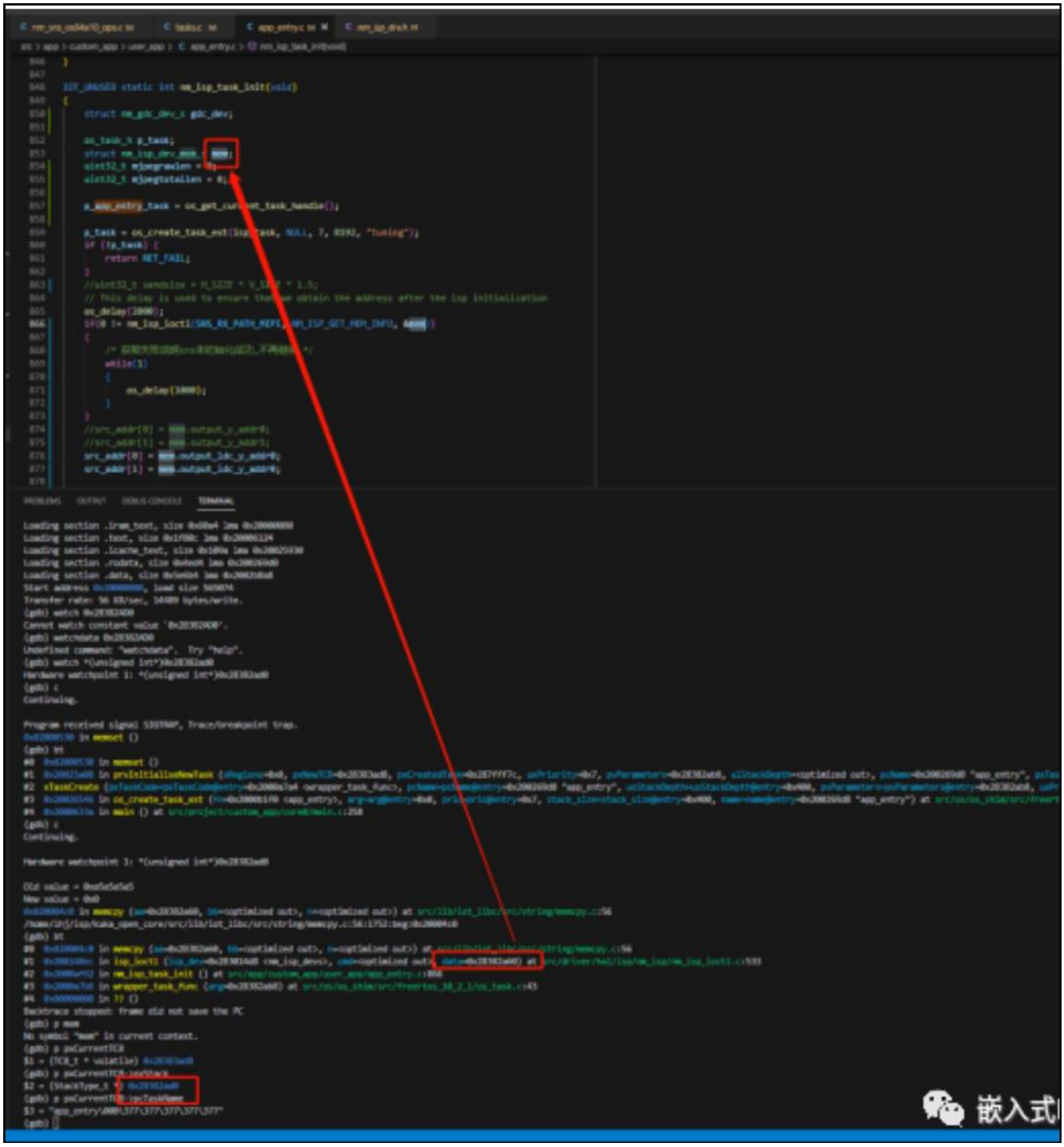
然後設定資料斷點

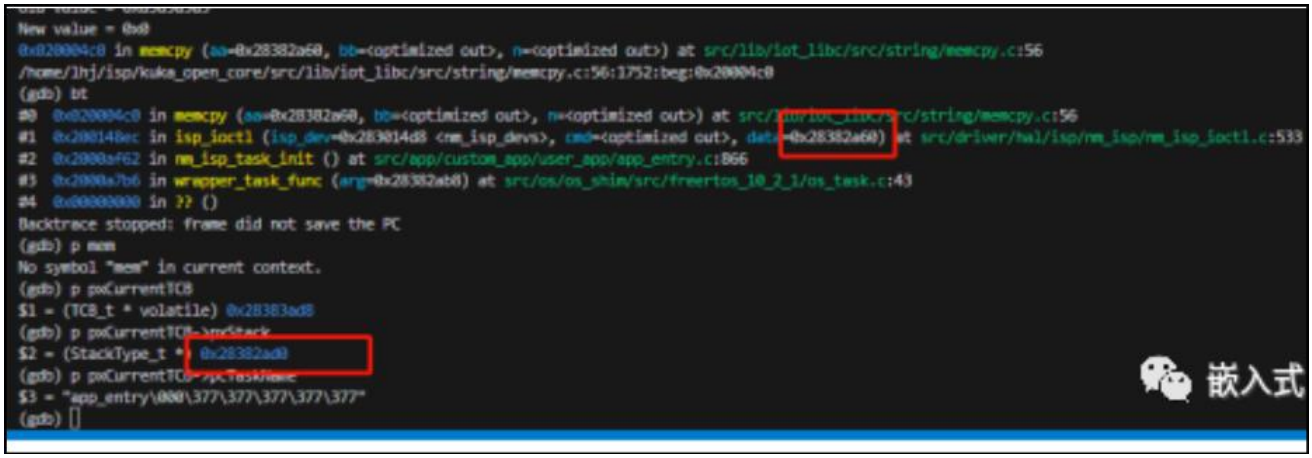
**watch *(unsigned int*)0x28382ad0**

運行，第一次停在棧初始化，第二次停在如下處

可以看到

任務棧的地址是 0x28382ad0 開始，但是臨時變數 mem 的地址是 0x28382a60 在棧前面去了，所以棧溢出了。




New value = 0x0
0x020004c0 in memcpy (aa=0x28382a60, bb=<optimized out>, n=<optimized out>) at src/lib/iot_libc/src/string/memcpy.c:56
/home/lhj/isp/kuka_open_core/src/lib/iot_libc/src/string/memcpy.c:56:1752:beg:0x20004c0
(gdb) bt
#0 0x020004c0 in memcpy (aa=0x28382a60, bb=<optimized out>, n=<optimized out>) at src/lib/iot_libc/src/string/memcpy.c:56
#1 0x20014bec in isp_ioctl (isp_dev=0x283014d8 <nm_isp_devs>, cmd=<optimized out>, data=0x28382a60) at src/driver/hal/isp/nm_isp/nm_isp_ioctl.c:533
#2 0x2000af62 in nm_isp_task_init () at src/app/custom_app/user_app/app_entry.c:866
#3 0x2000a7b6 in wrapper_task_func (arg=0x28382ab8) at src/os/os_shim/src/freertos_10_2_1/os_task.c:43
#4 0x00000000 in ?? ()
Backtrace stopped: frame did not save the PC
(gdb) p mem
No symbol "mem" in current context.
(gdb) p pxCurrentTCB
$1 = (TCB_t * volatile) 0x28383ad8
(gdb) p pxCurrentTCB->pxStack
$2 = (StackType_t *) 0x28382ad0
(gdb) p pxCurrentTCB->pcTaskName
$3 = "app_entry\000\377\377\377\377\377\377"
(gdb)

把栈改打即可